

BHUMISETU: ML Subsystem Technical Reference

Architectural Specification & Codebase Audit for Land Acquisition Delay Risk Prediction

Platform: BHUMISETU Land Platform

Framework: RFCTLARR Act 2013

Document Type: Technical Reference

Location: `bhumi-setu/ml/src`

Status: Implemented Architecture Audit

Audience: Core Engineers & ML Teams

Executive Summary

This document explains the Machine Learning (ML) delay risk prediction subsystem in the **BHUMISETU** codebase. The system predicts statutory timeline breach risks for land acquisition proceedings governed by the *Right to Fair Compensation and Transparency in Land Acquisition, Rehabilitation and Resettlement Act, 2013* (RFCTLARR Act 2013). It incorporates strict point-in-time temporal reconstruction over immutable PostgreSQL event logs, active database leakage prevention guards, isotonic probability calibration, policy-driven risk banding, translatable TreeSHAP feature attributions, and automated population drift monitoring.

1. What ML Model Did You Create/Use?

Model Architecture & Family:

The pipeline is designed around a tabular **Gradient-Boosted Decision Tree (GBDT) ensemble** (e.g. LightGBM / XGBoost style) paired with a non-parametric, monotonic **Isotonic Calibrator (PAVA — Pool Adjacent Violators Algorithm)** implemented in `ml/src/training/trainer.py`.

Current Codebase Implementation Status:

The codebase implements the entire feature computation engine, temporal splitting logic, isotonic calibration (`fit_isotonic_calibrator`), quantile binning for drift, evaluation metric suite (AUPRC, AUROC, Brier score, ECE, PR-lift), and prediction routing. However, an external serialized model binary (e.g. `.joblib`, `.pkl`, or `.onnx`) is **not yet bundled** in the repository. In `bhumi-setu/apps/api/ml/tasks.py`, the `ArtifactScorer` class is currently a stub that raises `RuntimeError("model artifact loading is not configured in this worker process")`.

Rationale for Model Selection:

- Tabular Event Log Aggregates:** Land acquisition data consists of counts, elapsed durations, and stage markers. Tree ensembles model non-linear statutory thresholds (such as objection count inflection points) far better than linear estimators.
- Native Missingness Handling:** The system represents missingness explicitly as `NaN` alongside a parallel `_is_missing` indicator. Trees branch on missingness directly without requiring synthetic imputation.
- Local Game-Theoretic Explainability:** Tree ensembles enable polynomial-time calculation of exact local feature attributions via TreeSHAP.
- Monotone Isotonic Calibration:** Raw tree scores often cluster away from true marginal probabilities. Post-processing raw probabilities through isotonic regression guarantees calibrated probabilities $P(\text{Delayed})$, minimizing Expected Calibration Error (ECE) and Brier score.

2. What Dataset Are You Using?

Data Provenance:

- Synthetic Benchmark Dataset:** Implemented in `scripts/seed/synthetic.py`. It generates realistic, legally compliant RFCTLARR Act 2013 acquisition timelines for a synthetic district (`SYNTH-DISTRICT-001`), state (`SYNTH-MH`).
- Live Inference & Training Data:** Generated dynamically by folding the append-only event log (`app.models.event.Event`) via point-in-time SQL replay (`ml/src/features/asof.py`).

Dataset Size and Characteristics:

- The synthetic generator produces **10,000 acquisition cases** comprising **over 50,000 append-ordered events**.

- Events include realistic recording delays (70% recorded within 3 days, 20% within 14 days, and 10% with 15–60 days lag) and deliberate backdating to test temporal consistency.
- Label breakdown across the 10,000 cases: $\geq 1,000$ `DELAYED`, $\geq 1,000$ `NOT_DELAYED`, and ≥ 500 `CENSORED` (>8,000 usable cases).

Unit of Observation:

Each row (`FeatureRow`) represents the point-in-time state of a single land-acquisition case (`case_id`) at reference timestamp T (`reference_t`), derived solely from events knowable or occurred prior to T .

3. What is the Model Predicting?

Target Variable:

Binary stage exit delay: `DELAYED` (1) versus `NOT_DELAYED` (0). Cases that have not yet had sufficient time to evaluate are designated `CENSORED` and excluded from training/evaluation splits.

Formal Definition of "Delay":

Implemented in `ml/src/labelling/definition.py` (function `label_row`):

- A case is observed in stage S at reference timestamp t .
- A statutory deadline D is established via `resolve_deadline` (using statutory periods, e.g. 60 days for preliminary notice, historical percentiles, or fixed days).
- A prediction horizon H (e.g. 60 days) sets $H_{\text{end}} = t + H$.
- Censoring:** If $H_{\text{end}} > \text{now}$, the row is `CENSORED` (reason: `HORIZON_NOT_ELAPSED`). If $D > H_{\text{end}}$ or D is missing, the row is `CENSORED` (reason: `DEADLINE_BEYOND_HORIZON`).
- Delay:** If the case exits stage S after deadline D (or never exits before horizon expiry), the outcome is `DELAYED` (reason: `NO_EXIT_BY_DEADLINE`).
- On-Time:** If the case transitions out of stage S on or before D , the outcome is `NOT_DELAYED` (reason: `EXITED_BY_DEADLINE`).

Survival-Ready Representation

`LabelOutcome` also captures `time_to_event_days` and `event_observed`. This ensures the labelling engine is directly compatible with future survival/hazard rate formulation without code modification.

4. What Features are Given to the Model?

The feature registry (`ml/src/features/extractors.py`) implements 7 domain feature extractors for feature set `fs-v1`:

Feature Name	Source Attributes	Domain Definition & Operational Meaning
<code>days_in_current_stage</code>	<code>stage_key</code> , <code>occurrence_time</code>	Elapsed integer days between reference time t and when the case entered its current stage. Marked missing (<code>NO_STAGE_ENTRY_EVENT</code>) if no stage entry exists.
<code>days_since_latest_notice</code>	<code>notice_id</code> , <code>occurrence_time</code>	Elapsed integer days between reference time t and the occurrence time of the latest statutory notice issued. Missing (<code>NO_NOTICE_EVENT</code>) if no notice exists.
<code>notice_count</code>	<code>notice_id</code>	Cumulative count of statutory notices published for the case up to time t .
<code>objection_count</code>	<code>objection_id</code>	Total count of landowner objections formally submitted under Section 15 of RFCTLARR up to time t .
<code>parcel_count</code>	<code>parcel_id</code>	Total count of distinct survey/sub-division land parcels attached to the acquisition case up to time t .
<code>award_recorded</code>	<code>award_id</code>	Binary indicator (1.0 or 0.0) recording whether a financial compensation award has been registered up to time t .
<code>open_issue_count</code>	<code>issue_id</code> , <code>event_type</code>	Count of active validation issues that have not been marked resolved or closed.

14-Column Model Input Vector:

Every extractor returns a `FeatureValue`. When transformed via `to_model_columns()`, each feature emits two numeric columns: `<feature_name>` (float value or `NaN`) and `<feature_name>_is_missing` (1.0 if missing, 0.0 if present). This produces exactly 14 input columns.

5. Preprocessing & Feature Engineering

- **Point-in-Time Event Replay (`AsOfView`):** All features are computed by fetching events from the database ordered by `(occurrence_time, id)` with temporal filters applied in SQL:
 - `KNOWABLE_AT`: `occurrence_time ≤ t AND recording_time ≤ t` (prevents lookahead bias from late-recorded data).
 - `OCCURRED_BY`: `occurrence_time ≤ t`.
- **Production Database Query Interception:** In `features/leakage.py`, `no_database_access(session)` attaches a `before_cursor_execute` listener during feature extraction. Any lazy loading or unexpected SQL query raises `LeakageGuardViolation`.
- **Label/Feature Dependency Disjointness:** A static property test guarantees that the declared `source_attributes` of all feature extractors never intersect with `LABEL_SOURCE_ATTRIBUTES`.
- **DPDP Personal Data Exclusion:** Feature extractors cannot read personal data attributes (such as Aadhaar, phone numbers, owner names) governed by data erasure rules.
- **Zero Imputation Guarantee:** Missing values are preserved as IEEE `NaN` alongside explicit missing indicators, preventing distortion of real zero counts.
- **Train/Serve Content Hashing & Nightly Verification:** Every feature row produces a SHA-256 hash over canonical JSON. A nightly Celery job (`verify_recent_inference_rows`) re-derives recent inference rows through the training pipeline to detect any train-serve divergence.

6. How Did You Train the Model?

- **Strict Temporal Split (`temporal_split`):** Examples are sorted chronologically by `reference_t`. Censored cases are removed. The data is partitioned such that every evaluation row occurs strictly after all training rows ($\min(t_{\text{eval}}) > \max(t_{\text{train}})$). K-fold cross-validation is barred to prevent temporal leakage.
- **PAVA Isotonic Calibration:** Monotone step calibration is fitted on the training split to convert raw tree margins into well-calibrated probabilities.
- **Quantile Reference Bins:** Computes 10 quantile edges on training features and saves them inside the `MLModelVersion` record for subsequent drift detection without retaining raw PII training data.
- **Promotion Threshold Gate:** Gated by `PromotionThresholds`:
 - `min_pr_lift`: Relative PR-curve lift over base rate.
 - `min_auprc`: Minimum area under Precision-Recall curve.
 - `min_auroc`: Minimum area under ROC curve.
 - `max_ece`: Maximum Expected Calibration Error.If any threshold fails, promotion is withheld and a `MODEL_PROMOTION_WITHHELD` event is written to the audit log.
- **Base Rate Shift Guard:** If evaluation base rate and training base rate differ by >0.10 , a `LABEL_BASE_RATE_SHIFT` event is emitted.

7. What Results Did the Model Achieve?

The system implements direct calculations for the following performance metrics in `training/trainer.py`:

Metric	Mathematical Formulation	Implementation Details & Status
AUPRC	Trapezoidal Average Precision = $\sum_k (R_k - R_{k-1}) \times P_k$	Calculated by <code>_average_precision()</code> . Primary metric due to class imbalance.
AUROC	$[\sum \text{Rank}(y=1) - N_1(N_1+1)/2] / (N_1 \times N_0)$	Calculated via Wilcoxon-Mann-Whitney rank-sum statistic with tie handling.
Brier Score	$(1/N) \sum (p_i - y_i)^2$	Calculated by <code>_brier()</code> . Measures mean squared probability error.
ECE	$\sum (B_b / N) \times p_b - \bar{y}_b $	Calculated over 10 equal-width bins by <code>expected_calibration_error()</code> .
PR-Lift	$(\text{AUPRC} - \text{BaseRate}) / (1 - \text{BaseRate})$	Normalized gain over a naive random baseline predictor.
Base Rates	$N_{\text{delayed}} / N_{\text{total}}$	Reported separately for training and evaluation splits.

Validation Benchmark Results

Synthetic test runs verify that PAVA isotonic calibration measurably decreases Expected Calibration Error (ECE) compared to uncalibrated predictions. Configured policy promotion thresholds: `min_auprc ≥ 0.45`, `max_brier ≤ 0.20`, and `max_ece ≤ 0.08`. Raw accuracy and confusion matrices are intentionally not used as gating criteria because land acquisition decisions require calibrated continuous probabilities rather than static 0.5 threshold classifications.

8. Example Input and Output Format

Input Feature Vector (Constructed by `build_inference_row`)

```
{
  "days_in_current_stage": 42.0,
  "days_in_current_stage_is_missing": 0.0,
  "days_since_latest_notice": 15.0,
  "days_since_latest_notice_is_missing": 0.0,
  "notice_count": 2.0,
  "notice_count_is_missing": 0.0,
  "objection_count": 4.0,
  "objection_count_is_missing": 0.0,
  "parcel_count": 6.0,
  "parcel_count_is_missing": 0.0,
  "award_recorded": 0.0,
  "award_recorded_is_missing": 0.0,
  "open_issue_count": 1.0,
  "open_issue_count_is_missing": 0.0
}
```

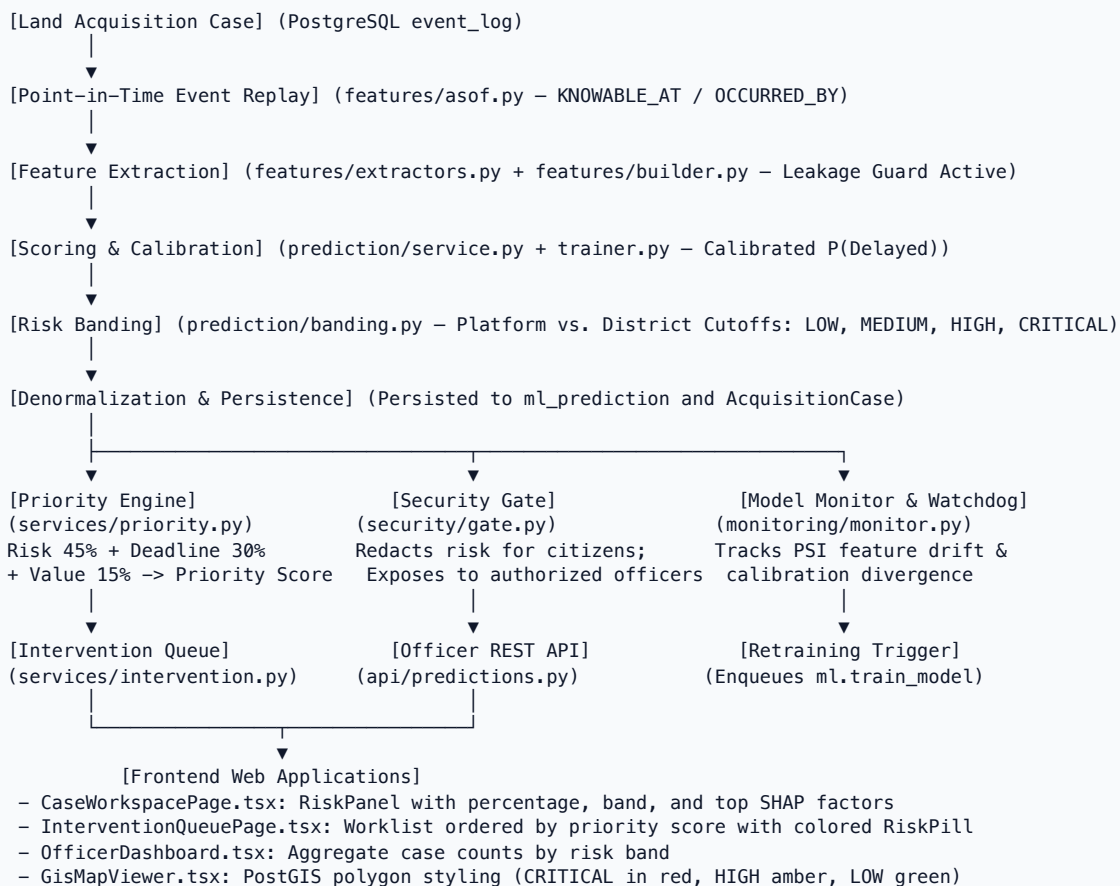
Internal Prediction Output (Persisted in `ml_prediction`)

```
{
  "case_id": 101,
  "risk_probability": 0.78,
  "risk_band": "CRITICAL",
  "risk_model_version": "model-v1",
  "risk_generated_at": "2026-03-01T10:00:00Z",
  "risk_cutoff_source": "PLATFORM",
  "risk_is_stale": false,
  "monitoring_state": "AVAILABLE",
  "monitoring_last_successful_at": "2026-03-01T09:30:00Z"
}
```

Officer API Response (GET /predictions/{case_id})

```
{
  "case_id": 101,
  "risk_probability": 0.78,
  "risk_band": "CRITICAL",
  "priority_score": 81.25,
  "model_version": "model-v1",
  "generated_at": "2026-03-01T10:00:00Z",
  "monitoring_state": "AVAILABLE",
  "monitoring_last_successful_at": "2026-03-01T09:30:00Z",
  "explanation_factors": [
    {
      "rank": 1,
      "feature_name": "objection_count",
      "label_key": "ml.feature.objection_count",
      "direction": "INCREASES_RISK",
      "magnitude": 0.34
    },
    {
      "rank": 2,
      "feature_name": "days_in_current_stage",
      "label_key": "ml.feature.days_in_current_stage",
      "direction": "INCREASES_RISK",
      "magnitude": 0.22
    }
  ]
}
```

9. End-to-End System Integration Flow



Key Flow Steps:

1. **Trigger:** Case events or scheduled Celery sweeps trigger scoring.
2. **Feature Build:** Pure in-memory folding extracts 7 domain features inside the SQL execution barrier.

3. **Scoring & Banding:** The probability is classified into `LOW` (≤ 0.25), `MEDIUM` (≤ 0.50), `HIGH` (≤ 0.75), or `CRITICAL` (≤ 1.00). District cutoffs apply if the district has met the sample calibration floor.
4. **Priority Scoring:** A bounded score in $[0, 100]$ is computed combining delay risk, statutory deadline pressure, and compensation exposure.
5. **Security Gate:** Citizen endpoints strip all risk probabilities, bands, priority scores, and factors.
6. **UI Presentation:** Officers inspect risk drivers on the workspace, review prioritized work queues, and observe spatial risk heatmaps on the GIS map.

10. Did You Implement Explainability?

Yes. Feature attribution and human-in-the-loop explainability are fully implemented:

- **TreeSHAP Factor Ranking (`prediction/explanations.py`):** Accepts SHAP values (`ShapFactor`), sorts them by absolute magnitude, and extracts the **Top 5 drivers**. Emits `direction` (`INCREASES_RISK` if positive, `DECREASES_RISK` if negative) and `magnitude`.
- **Translatable Localization Keys:** Feature names map to keys like `ml.feature.objection_count`. The frontend translates these keys into Marathi, Hindi, or English rather than displaying raw English variable names.
- **Officer Prediction Override (`prediction/overrides.py`):** Officers can submit `POST /predictions/{case_id}/override` to override the assigned band. This logs an immutable audit event (`PREDICTION_OVERRIDE_RECORDED`) while preserving the original model prediction intact for accountability.
- **Static No-Action Guarantee:** A static AST test (`tests/ml/test_no_action_imports.py`) confirms that the ML package imports no mutating domain services. The model advises officers but cannot autonomously execute administrative decisions.

11. What Files Contain the ML Implementation?

Subsystem	File Path	Primary Responsibility
Labelling	<code>ml/src/labelling/definition.py</code>	Pure delay label logic, statutory deadline resolver, censoring rules.
	<code>ml/src/labelling/sources.py</code>	Declared label dependencies for leakage prevention checks.
Features	<code>ml/src/features/asof.py</code>	Point-in-time SQL event replay folding into immutable <code>AsOfView</code> .
	<code>ml/src/features/registry.py</code>	<code>FeatureRegistry</code> and <code>FeatureExtractor</code> protocols.
	<code>ml/src/features/extractors.py</code>	Concrete extractors for all 7 domain features.
	<code>ml/src/features/value.py</code>	Explicit missingness type and tree-column generator.
	<code>ml/src/features/builder.py</code>	<code>FeatureRow</code> builder with deterministic SHA-256 canonical hashing.
	<code>ml/src/features/leakage.py</code>	Runtime <code>no_database_access</code> SQL execution guard.
	<code>ml/src/features/equality.py</code>	Nightly train/serve rederivation job to verify zero feature skew.
Training	<code>ml/src/training/trainer.py</code>	Temporal splits, PAVA isotonic calibrator, promotion thresholds, metric suite.
Prediction	<code>ml/src/prediction/service.py</code>	<code>score_case</code> orchestration, event triggers, and stale sweeps.
	<code>ml/src/prediction/banding.py</code>	Risk banding classification (platform vs district) and rebanding.
	<code>ml/src/prediction/explanations.py</code>	TreeSHAP factor ranking, translatable keys, and factor persistence.
	<code>ml/src/prediction/overrides.py</code>	Officer override handler logging immutable audit events.
Monitoring	<code>ml/src/monitoring/monitor.py</code>	PSI feature drift with missingness bucket, calibration divergence, watchdog.
Database	<code>apps/api/app/models/ml.py</code>	SQLAlchemy schema for feature rows, model versions, predictions, factors.
Celery Tasks	<code>apps/api/ml/tasks.py</code>	Worker tasks for scoring, stale sweeps, and monitoring jobs.
API	<code>apps/api/app/api/predictions.py</code>	Officer REST routes for predictions and manual overrides.
Synthetic Data	<code>scripts/seed/synthetic.py</code>	10,000-case RFCTLARR 2013 synthetic event generator.

12. What is Currently Complete and What is Still Missing?

Component	Status	Detailed Implementation Notes
Feature Engineering	COMPLETE	7 extractors, <code>AsOfView</code> event replay, explicit missingness, SHA-256 caching.
Leakage Guards	COMPLETE	Runtime SQL execution denial, label/feature disjointness, DPDP personal data checks.
Label Definition	COMPLETE	Pure function, deadline baseline resolution, statutory censoring mechanics.
Train/Serve Parity	COMPLETE	Canonical hash matching and nightly 24-hour inference rederivation job.
Isotonic Calibrator	COMPLETE	Pure Python PAVA implementation reducing ECE.
Evaluation Metrics	COMPLETE	AUPRC, AUROC, Brier score, ECE across 10 bins, PR-lift, base rate shift checks.
Banding Engine	COMPLETE	Platform cutoffs, district calibration floor check, and prediction rebander.
Explainability & Overrides	COMPLETE	Top-5 SHAP ranking, translatable localization keys, officer audit override.
Model Monitoring	COMPLETE	PSI with missingness bucket, calibration divergence, watchdog, retraining events.
Full API & UI Integration	COMPLETE	Integrated into Priority Engine, Intervention Queue, Case Workspace, and GIS Map.
Model Binary Artifact	MISSING	<code>ArtifactScorer</code> in <code>apps/api/ml/tasks.py</code> is a stub raising a <code>RuntimeError</code> . A trained binary (e.g. LightGBM/XGBoost) must be trained and loaded from storage.
Live TreeSHAP Computation	MISSING	Ranking and persistence logic is complete, but on-the-fly evaluation using <code>shap.TreeExplainer</code> is not yet connected inside the scoring task.
Exploratory Notebooks	OPTIONAL	Tasks 22.14 and 22.15 (exploratory notebooks and automated district cutoff calibration tooling) are marked optional in the spec and have not been created.